AI Research Assistant – Project

Day - 18

1. Objective

The central goal for Day 18 was to implement the core intelligence of the application: the multi-source summarization engine. This involved integrating with a powerful external Large Language Model (LLM), Google's Gemini, and developing the backend logic to process three distinct types of input—a user-defined topic, a webpage URL, and an uploaded PDF document.

2. Theoretical Concepts Applied

- **Application Programming Interface (API):** An API is a set of rules and protocols that allows different software applications to communicate with each other. In our project, we used the Google Gemini API to send requests from our Flask server to Google's powerful AI models. Our application sends a **prompt** (a set of instructions and text) to the API endpoint, and Google's servers process it and send back a **response** containing the AI-generated summary. This allows us to leverage massive computational power without needing to host the model ourselves.
- **Large Language Models (LLMs):** An LLM, like Google Gemini, is a type of artificial intelligence trained on vast amounts of text data. It learns the patterns, structures, and nuances of human language, allowing it to understand context and generate coherent, human-like text. The quality and style of its output are heavily influenced by the instructions it receives.
- **Prompt Engineering:** This is the art and science of designing effective inputs (prompts) to guide an LLM towards a desired output. A well-crafted prompt is crucial for getting high-quality results. On Day 3, we engineered a detailed_prompt_template that instructs the AI to generate a "comprehensive summary structured primarily with detailed bullet points," which guides the model to produce a more organized and useful result than a simple instruction like "summarize this."
- **Web Scraping & HTML Parsing:** To summarize a URL, the application must first extract its text content. This process, known as web scraping, involves two steps:
    1. **Fetching:** Using the requests library, our server sends an HTTP GET request to the target URL to download its raw HTML source code.
    2. **Parsing:** The raw HTML is a structured document full of tags (<p>, <h1>, , etc.). We use the BeautifulSoup library, a powerful HTML parser, to navigate this structure and intelligently extract only the human-readable text content, discarding irrelevant code, scripts, and styling.

3. Key Activities & Implementation Details

- **Secure API Key Management:**
    - The python-dotenv library was installed.
    - A .env file was created in the root project directory to store the GOOGLE_API_KEY. This file is a standard way to manage secret credentials. It is included in the .gitignore file to ensure it is never committed to public version control, which is a critical security practice.
    - The load_dotenv() function was called at the start of app.py to load these secrets into the application's environment variables.
- **LangChain Integration:**
    - The langchain_google_genai library was installed to serve as a high-level interface to the Gemini API, simplifying the process of authenticating and sending prompts.
- **Modular Utility Functions:**
    - To adhere to the "Separation of Concerns" principle, the logic for external interactions was moved into a utils/ directory.
    - **utils/gemini.py:** A function gemini_generate_summary() was created. It takes a fully formatted prompt, initializes the ChatGoogleGenerativeAI model from LangChain, and returns the AI-generated text.
    - **utils/extract.py:** Two functions were created:
        - extract_text_from_url(): Uses requests and BeautifulSoup to perform the web scraping process.
        - extract_text_from_pdf(): Uses the PyPDF2 library to open and read the text content from an uploaded PDF file.
- **Main Processing Logic (/process route):**
    - The primary backend route was developed to orchestrate the summarization process.
    - It uses conditional logic (if/elif/else) to check the input_type received from the user's form submission.
    - **Topic Logic:** If input_type is "topic," it takes the user's text directly.
    - **URL Logic:** If "url," it calls extract_text_from_url() to get the content.
    - **PDF Logic:** If "pdf," it handles the file upload using request.files, saves the file to an uploads/ directory, and then calls extract_text_from_pdf().
    - In all cases, the extracted text is formatted into the detailed_prompt_template and passed to gemini_generate_summary().

- **Database Persistence:**
    - The database schema was extended by creating a history table. This table is designed to store the username, the topic (which includes the URL or filename), the generated summary, and a timestamp for every request, providing a persistent record of user activity.

4. Outcome

At the conclusion of Day 18, the AI Research Assistant is now a fully-fledged intelligent application. It successfully integrates with a powerful external LLM, processes multiple types of user input, and securely stores the results. The core value proposition of the project has been realized, laying the groundwork for more advanced interactive features.